



**FACULTAD DE SISTEMAS Y
TELECOMUNICACIONES**

CARRERA DE SOFTWARE



CARRERA DE **SOFTWARE**

FACULTAD DE SISTEMAS Y TELECOMUNICACIONES

INGENIERÍA EN SOFTWARE

MATERIA:

PROGRAMACIÓN FUNCIONAL

DOCENTE:

Ing. ANTHONY ABRAHAN PACHAY ESPINOZA

NOMBRE:

DAYWIN JOAN SALVATIERRA COMANICHO

SEMESTRE:

QUINTO SEMESTRE 5/1

PERÍODO:

2026-2

Taller de programación funcional en Python

Funciones de orden superior, closures y lambdas

El taller reúne veinte ejercicios organizados en cuatro niveles. En cada caso se presenta una breve explicación y una imagen con la implementación, las pruebas con `print()` y la salida obtenida al ejecutar el archivo en Python.

Repositorio público

<https://github.com/DaywinJC/taller-programacion-funcional>

Organización y ejecución

Nivel 1: closures con transformación y reglas dinámicas.

Nivel 2: contadores, acumuladores, promedios y estados.

Nivel 3: colecciones, composición, historial y auditoría.

Nivel 4: validación, memoización, pipelines, eventos y consultas.

Para ejecutar todos los ejemplos, se usa `python ejecutar_todo.py`. Para comprobar los resultados y casos límite, se usa `python verificar.py`. El proyecto requiere Python 3.10 o posterior y no necesita paquetes externos.

Pruebas de ejecución

Los veinte archivos finalizaron sin errores. Las imágenes reúnen el texto del código y la salida capturada de cada proceso de Python. La duración de la auditoría puede variar entre ejecuciones. Los ejemplos aleatorios tienen una semilla fija para que sus resultados sean reproducibles.

Además de las pruebas visibles con `print()`, se comprobaron el reinicio del limitador, la independencia entre closures, la sustitución de entradas en la caché LRU y el rechazo de parámetros inválidos.

1 Formateadores con transformación

El closure conserva el prefijo y la función que transforma el texto. Así se pueden crear varios formateadores sin repetir la misma lógica. En las pruebas, uno convierte a mayúsculas, otro invierte el texto y el último limpia los espacios del nombre.

Ejercicio 01 | Código Python

```
def crear_formateador(prefijo, fn_transformacion):
    def formatear(texto):
        return prefijo + fn_transformacion(texto)
    return formatear

if __name__ == "__main__":
    formateador_mayus = crear_formateador("[LOG] ", lambda t: t.upper())
    formateador_invertido = crear_formateador(">> ", lambda t: t[::-1])

    print(formateador_mayus("sistema iniciado"))
    print(formateador_invertido("python"))
    print(crear_formateador("Usuario: ", lambda t: t.strip().title())(" ana lopez "))
```

Salida capturada al ejecutar el archivo en Python

```
[LOG] SISTEMA INICIADO
>> nohtyp
Usuario: Ana Lopez
```

El prefijo se añade después de aplicar la transformación.

2 Operador con un factor fijo

La función devuelta recibe un valor y lo combina con el factor capturado. La lambda decide si se multiplica, se calcula una potencia o se resta. El mismo operador también se puede aplicar a todos los elementos de una lista.

Ejercicio 02 | Código Python

```
def crear_operador(factor, operacion_lambda):
    def operar(valor):
        return operacion_lambda(valor, factor)
    return operar

if __name__ == "__main__":
    duplicar = crear_operador(2, lambda x, f: x * f)
    elevar = crear_operador(3, lambda x, f: x ** f)
    restar_factor = crear_operador(5, lambda x, f: x - f)

    print(duplicar(10))
    print(elevar(2))
    print(restar_factor(20))
    print([duplicar(v) for v in [1, 2, 3, 4]])
```

Salida capturada al ejecutar el archivo en Python

```
20
8
15
[2, 4, 6, 8]
```

El factor pertenece a cada closure; crear otro operador no modifica el anterior.

3 Descuento con una regla dinámica

La condición determina qué precios reciben el descuento. Para precios mayores de 100 se aplica un 15 %; en el otro ejemplo se descuenta un 5 % a los precios pares. Los valores que no cumplen la regla se devuelven sin cambios.

Ejercicio 03 | Código Python

```
def crear_descuento_dinamico(regla_condicional_lambda, porcentaje=0.10):
    def aplicar(precio):
        if regla_condicional_lambda(precio):
            return round(precio * (1 - porcentaje), 2)
        return precio
    return aplicar

if __name__ == "__main__":
    descuento_mayor_100 = crear_descuento_dinamico(lambda p: p > 100, porcentaje=0.15)
    descuento_par = crear_descuento_dinamico(lambda p: p % 2 == 0, porcentaje=0.05)

    print(descuento_mayor_100(150))
    print(descuento_mayor_100(50))
    print(descuento_par(80))
    print(descuento_par(81))
```

Salida capturada al ejecutar el archivo en Python

```
127.5
50
76.0
81
```

Con 150 el resultado es 127.5; con 50 no se aplica el descuento.

4 Generador de nombres con sufijos

Cada llamada incrementa un contador privado y lo entrega a la lambda junto con el nombre base. El patrón decide cómo presentar el número. Los dos generadores de la prueba mantienen contadores independientes.

Ejercicio 04 | Código Python

```
def crear_generador_sufijos(patron_lambda):
    contador = {"n": 0}

    def generar(nombre_base):
        contador["n"] += 1
        return patron_lambda(nombre_base, contador["n"])
    return generar

if __name__ == "__main__":
    serial_guiion = crear_generador_sufijos(lambda nombre, n: f"{nombre}-{n:03d}")
    serial_parentesis = crear_generador_sufijos(lambda nombre, n: f"{nombre}({n})")

    print(serial_guiion("reporte"))
    print(serial_guiion("reporte"))
    print(serial_guiion("reporte"))
    print(serial_parentesis("foto"))
    print(serial_parentesis("foto"))
```

Salida capturada al ejecutar el archivo en Python

```
reporte-001
reporte-002
reporte-003
foto(1)
foto(2)
```

Para evitar nombres repetidos, el patrón debe incluir el contador.

5 Conversión de divisas con comisión

Primero se multiplica el monto por la tasa y luego se calcula el recargo con la lambda. La primera regla cobra un 2 %; la segunda usa una comisión fija que depende del importe convertido. Las tasas son datos de ejemplo.

Ejercicio 05 | Código Python

```
def crear_convertor(tasa, margen_lambda):
    def convertir(monto):
        convertido = monto * tasa
        comision = margen_lambda(convertido)
        return round(convertido + comision, 2)
    return convertir

if __name__ == "__main__":
    usd_a_eur = crear_convertor(0.92, lambda m: m * 0.02)
    usd_a_pen = crear_convertor(3.75, lambda m: 5 if m > 200 else 1.5)

    print(usd_a_eur(100))
    print(usd_a_pen(50))
    print(usd_a_pen(300))
```

Salida capturada al ejecutar el archivo en Python

```
93.84
189.0
1130.0
```

El resultado representa el importe convertido más la comisión, redondeado a dos decimales.

6 Contador con actualización variable

La variable cuenta empieza en cero. La función interna usa nonlocal para actualizarla con el resultado de fn_paso. En esta solución, la lambda calcula el nuevo valor completo: una regla produce 1, 3 y 7; la otra avanza de tres en tres.

Ejercicio 06 | Código Python

```
def crear_contador_paso(fn_paso):
    cuenta = 0

    def siguiente():
        nonlocal cuenta
        cuenta = fn_paso(cuenta)
        return cuenta
    return siguiente

if __name__ == "__main__":
    contador_doble = crear_contador_paso(lambda actual: actual * 2 + 1)
    contador_de_tres = crear_contador_paso(lambda actual: actual + 3)

    print(contador_doble())
    print(contador_doble())
    print(contador_doble())
    print(contador_de_tres())
    print(contador_de_tres())
```

Salida capturada al ejecutar el archivo en Python

```
1
3
7
3
6
```

El estado se conserva aunque la llamada que creó el contador ya terminó.

7 Acumulador con filtro de aceptación

El acumulador solo añade un dato cuando la condición devuelve True. Una prueba acepta números positivos y otra acepta números pares. Un valor rechazado deja el total anterior intacto.

Ejercicio 07 | Código Python

```
def crear_acumulador_validado(criterio_lambda):
    total = 0

    def acumular(valor):
        nonlocal total
        if criterio_lambda(valor):
            total += valor
        return total
    return acumular

if __name__ == "__main__":
    acumulador_positivos = crear_acumulador_validado(lambda v: v > 0)
    acumulador_pares = crear_acumulador_validado(lambda v: v % 2 == 0)

    print(acumulador_positivos(10))
    print(acumulador_positivos(-5))
    print(acumulador_positivos(20))
    print(acumulador_pares(3))
    print(acumulador_pares(4))
    print(acumulador_pares(8))
```

Salida capturada al ejecutar el archivo en Python

```
10
10
30
0
4
12
```

Después de recibir -5, el primer acumulador sigue mostrando 10.

8 Promedio sin valores fuera de rango

Los datos se guardan en una lista privada. Antes de calcular el promedio se conservan únicamente los que cumplen el filtro. La prueba acepta el rango de 0 a 100, por eso el valor 999 no altera el promedio.

Ejercicio 08 | Código Python

```
def crear_promediador_filtrado(filtro_ruido_lambda):
    datos = []

    def agregar(valor):
        datos.append(valor)
        limpios = [d for d in datos if filtro_ruido_lambda(d)]
        if not limpios:
            return 0
        return round(sum(limpios) / len(limpios), 2)
    return agregar

if __name__ == "__main__":
    # descarta valores fuera del rango 0-100
    promedio_sensor = crear_promediador_filtrado(lambda v: 0 <= v <= 100)

    print(promedio_sensor(20))
    print(promedio_sensor(30))
    print(promedio_sensor(999)) # ruido, se descarta
    print(promedio_sensor(40))
```

Salida capturada al ejecutar el archivo en Python

```
20.0
25.0
25.0
30.0
```

El promedio final de 20, 30 y 40 es 30.0. Si no hay datos válidos, se devuelve 0.

9 Limitador de intentos con reinicio

El closure cuenta las llamadas y permite hasta dos intentos en el ejemplo. Cuando se supera el máximo, ejecuta la alerta y devuelve False. El método reset restablece el contador para iniciar una nueva secuencia.

Ejercicio 09 | Código Python

```
def crear_limitador_avanzado(max_intentos, fn_alerta):
    if not isinstance(max_intentos, int) or isinstance(max_intentos, bool) or
    max_intentos < 1:
        raise ValueError("max_intentos debe ser un entero positivo")
    intentos = 0

    def ejecutar():
        nonlocal intentos
        intentos += 1
        if intentos > max_intentos:
            fn_alerta(intentos)
            return False
        return True

    def reset():
        nonlocal intentos
        intentos = 0

    ejecutar.reset = reset
    return ejecutar

if __name__ == "__main__":
    alerta = lambda n: print(f"ALERTA: limite excedido en intento {n}")
    limitador = crear_limitador_avanzado(2, alerta)

    print(limitador())
    print(limitador())
    print(limitador())
    limitador.reset()
    print(limitador())
```

Salida capturada al ejecutar el archivo en Python

```
True
True
ALERTA: limite excedido en intento 3
False
True
```

Se limita el número de intentos desde el último reinicio; no se usa una ventana de tiempo.

10 Conmutador de estados

El índice avanza una posición en cada llamada. La operación módulo permite volver al principio cuando se alcanza el último estado. La secuencia se copia al crear el closure, de modo que modificar la lista original no altera el ciclo.

Ejercicio 10 | Código Python

```
def crear_conmutador(lista_estados):
    estados = tuple(lista_estados)
    if not estados:
        raise ValueError("Se requiere al menos un estado")
    indice = -1

    def siguiente_estado():
        nonlocal indice
        indice = (indice + 1) % len(estados)
        return estados[indice]
    return siguiente_estado

if __name__ == "__main__":
    semaforo = crear_conmutador(["rojo", "amarillo", "verde"])

    print(semaforo())
    print(semaforo())
    print(semaforo())
    print(semaforo()) # vuelve a rojo
```

Salida capturada al ejecutar el archivo en Python

```
rojo
amarillo
verde
rojo
```

La cuarta llamada vuelve a rojo. Una lista vacía se rechaza con ValueError.

11 Filtrado y transformación de una colección

`filter` selecciona los valores que cumplen el predicado y `map` aplica la transformación a esos valores. La primera prueba obtiene cuadrados de números pares. La segunda multiplica por diez los números mayores de cinco.

Ejercicio 11 | Código Python

```
def procesar_coleccion(lista, fn_predicado, fn_transformacion):
    return list(map(fn_transformacion, filter(fn_predicado, lista)))

if __name__ == "__main__":
    numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

    print(procesar_coleccion(numeros, lambda n: n % 2 == 0, lambda n: n ** 2))
    print(procesar_coleccion(numeros, lambda n: n > 5, lambda n: n * 10))
```

Salida capturada al ejecutar el archivo en Python

```
[4, 16, 36, 64, 100]
[60, 70, 80, 90, 100]
```

El filtrado ocurre antes del mapeo; los elementos rechazados no se transforman.

12 Agrupación mediante una clave

La lambda calcula la clave de cada elemento y el diccionario acumula los elementos de una misma clave en una lista. En el primer ejemplo se agrupan estudiantes por curso. En el segundo se separan números pares e impares.

Ejercicio 12 | Código Python

```
def agrupar_por(lista, fn_clave):
    resultado = {}
    for elemento in lista:
        clave = fn_clave(elemento)
        resultado.setdefault(clave, []).append(elemento)
    return resultado

if __name__ == "__main__":
    estudiantes = [
        {"nombre": "Ana", "curso": "A"},
        {"nombre": "Luis", "curso": "B"},
        {"nombre": "Marta", "curso": "A"},
        {"nombre": "Juan", "curso": "C"},
    ]

    print(agrupar_por(estudiantes, lambda e: e["curso"]))

    numeros = [1, 2, 3, 4, 5, 6, 7, 8]
    print(agrupar_por(numeros, lambda n: "par" if n % 2 == 0 else "impar"))
```

Salida capturada al ejecutar el archivo en Python

```
{'A': [{'nombre': 'Ana', 'curso': 'A'}, {'nombre': 'Marta', 'curso': 'A'}], 'B':
[{'nombre': 'Luis', 'curso': 'B'}], 'C': [{'nombre': 'Juan', 'curso': 'C'}]}
{'impar': [1, 3, 5, 7], 'par': [2, 4, 6, 8]}
```

La clave debe poder usarse en un diccionario, como una cadena o un número.

13 Ejecución repetida e historial

Cada llamada al closure ejecuta la tarea N veces y añade los resultados a un historial privado. El método historial permite consultar lo acumulado sin volver a ejecutar la tarea. La prueba del dado usa una semilla para repetir sus resultados.

Ejercicio 13 | Código Python

```
def ejecutar_y_rastrear(fn_tarea, n):
    if not isinstance(n, int) or isinstance(n, bool) or n < 0:
        raise ValueError("n debe ser un entero no negativo")
    historial = []

    def ejecutar_todo(*args, **kwargs):
        for _ in range(n):
            historial.append(fn_tarea(*args, **kwargs))
        return list(historial)

    def obtener_historial():
        return list(historial)

    ejecutar_todo.historial = obtener_historial
    return ejecutar_todo

if __name__ == "__main__":
    import random
    random.seed(1)

    lanzar_dado = ejecutar_y_rastrear(lambda: random.randint(1, 6), 5)
    print(lanzar_dado())
    print(lanzar_dado.historial())

    contador_llamadas = ejecutar_y_rastrear(lambda x: x * 2, 3)
    print(contador_llamadas(4))
```

Salida capturada al ejecutar el archivo en Python

```
[2, 5, 1, 3, 1]
[2, 5, 1, 3, 1]
[8, 8, 8]
```

El historial se devuelve como una copia de la lista para evitar cambios externos en su estructura.

14 Composición de dos funciones

La función compuesta aplica primero g y después f . El orden importa: sumar uno y duplicar produce 12 para el valor 5, mientras que duplicar y sumar uno produce 11. La misma idea se aplica a las transformaciones de texto.

Ejercicio 14 | Código Python

```
def componer_dos(f, g):
    def compuesta(x):
        return f(g(x))
    return compuesta

if __name__ == "__main__":
    incrementar = lambda x: x + 1
    duplicar = lambda x: x * 2

    incrementar_luego_duplicar = componer_dos(duplicar, incrementar)
    duplicar_luego_incrementar = componer_dos(incrementar, duplicar)

    print(incrementar_luego_duplicar(5)) # (5+1)*2 = 12
    print(duplicar_luego_incrementar(5)) # (5*2)+1 = 11

    mayusculas_e_invertido = componer_dos(lambda t: t[::-1], lambda t: t.upper())
    print(mayusculas_e_invertido("python"))
```

Salida capturada al ejecutar el archivo en Python

```
12
11
NOHTYP
```

La expresión que resume el comportamiento es $f(g(x))$.

15 Tiempo de ejecución y auditoría

La envoltura toma una marca de tiempo antes y después de ejecutar la función. Luego envía al logger el nombre, la duración y el resultado. En la prueba se calcula la suma de cuadrados de los enteros entre 0 y 99999.

Ejercicio 15 | Código Python

```
import time

def auditar_ejecucion(fn_objetivo, fn_logger):
    def envoltura(*args, **kwargs):
        inicio = time.perf_counter()
        resultado = fn_objetivo(*args, **kwargs)
        duracion = time.perf_counter() - inicio
        fn_logger(fn_objetivo.__name__, duracion, resultado)
        return resultado
    return envoltura

if __name__ == "__main__":
    logger_simple = lambda nombre, dur, res: print(
        f"[AUDITORIA] {nombre} -> resultado={res} tiempo={dur:.6f}s"
    )

    def tarea_pesada(n):
        return sum(i * i for i in range(n))

    tarea_auditada = auditar_ejecucion(tarea_pesada, logger_simple)
    print(tarea_auditada(100000))
```

Salida capturada al ejecutar el archivo en Python

```
[AUDITORIA] tarea_pesada -> resultado=333328333350000 tiempo=0.014985s
333328333350000
```

La duración puede cambiar en cada ejecución. Si la tarea falla, su excepción se propaga.

16 Validación con varias reglas

all evalúa los criterios y acepta el objeto solo si todos se cumplen. El usuario de ejemplo necesita nombre, al menos 18 años y un correo que contenga @. El segundo usuario se rechaza porque tiene 15 años.

Ejercicio 16 | Código Python

```
def crear_validador_multiple(*lambdas_criterios):
    def validar(objeto):
        return all(criterio(objeto) for criterio in lambdas_criterios)
    return validar

# Alias que conserva exactamente el nombre del enunciado.
crear_validador_múltiple = crear_validador_multiple

if __name__ == "__main__":
    es_valido_usuario = crear_validador_multiple(
        lambda u: len(u.get("nombre", "")) > 0,
        lambda u: u.get("edad", 0) >= 18,
        lambda u: "@" in u.get("email", ""),
    )

    usuario_ok = {"nombre": "Ana", "edad": 25, "email": "ana@mail.com"}
    usuario_menor = {"nombre": "Luis", "edad": 15, "email": "luis@mail.com"}

    print(es_valido_usuario(usuario_ok))
    print(es_valido_usuario(usuario_menor))
```

Salida capturada al ejecutar el archivo en Python

```
True
False
```

La comprobación de @ es una regla didáctica, no una validación completa de direcciones de correo.

17 Memoización con capacidad limitada

La caché guarda resultados para reutilizarlos cuando se repiten los argumentos. Su política es LRU: se elimina la entrada que lleva más tiempo sin utilizarse. Consultar otra vez el 2 lo marca como reciente, de modo que al insertar el 4 se elimina el 3.

Ejercicio 17 | Código Python

```
from collections import OrderedDict

def memoizar_avanzado(fn_costosa, max_items):
    if not isinstance(max_items, int) or isinstance(max_items, bool) or max_items < 1:
        raise ValueError("max_items debe ser un entero positivo")
    cache = OrderedDict()

    def envoltura(*args):
        if args in cache:
            cache.move_to_end(args)
            return cache[args]
        resultado = fn_costosa(*args)
        cache[args] = resultado
        if len(cache) > max_items:
            cache.popitem(last=False)
        return resultado

    envoltura.cache_info = lambda: dict(cache)
    return envoltura

if __name__ == "__main__":
    def cuadrado_costoso(n):
        print(f" (calculando cuadrado de {n})")
        return n * n

    cuadrado_cacheado = memoizar_avanzado(cuadrado_costoso, max_items=2)

    print(cuadrado_cacheado(2))
    print(cuadrado_cacheado(3))
    print(cuadrado_cacheado(2)) # desde cache
    print(cuadrado_cacheado(4)) # descarta el 3 (LRU)
    print(cuadrado_cacheado.cache_info())
```

Salida capturada al ejecutar el archivo en Python

```
(calculando cuadrado de 2)
4
(calculando cuadrado de 3)
9
4
(calculando cuadrado de 4)
16
{(2,): 4, (4,): 16}
```

Los mensajes de cálculo de la prueba permiten distinguir un cálculo nuevo de una lectura de caché.

18 Pipeline de transformaciones

El dato inicial pasa por las funciones en el orden en que se declararon. El ejemplo de texto elimina espacios exteriores, convierte a minúsculas y sustituye espacios por guiones bajos. El ejemplo numérico suma, multiplica y resta.

Ejercicio 18 | Código Python

```
def crear_pipeline(*funciones_transformacion):
    def ejecutar(dato_inicial):
        resultado = dato_inicial
        for funcion in funciones_transformacion:
            resultado = funcion(resultado)
        return resultado
    return ejecutar

if __name__ == "__main__":
    pipeline_texto = crear_pipeline(
        lambda t: t.strip(),
        lambda t: t.lower(),
        lambda t: t.replace(" ", "_"),
    )
    print(pipeline_texto("  Hola Mundo Python  "))

    pipeline_numeros = crear_pipeline(
        lambda n: n + 10,
        lambda n: n * 2,
        lambda n: n - 5,
    )
    print(pipeline_numeros(5))
```

Salida capturada al ejecutar el archivo en Python

```
hola_mundo_python
25
```

Un pipeline sin funciones devuelve el dato inicial sin modificarlo.

19 Publicación y suscripción de eventos

El gestor es un closure que conserva los suscriptores de cada evento. La acción suscribirse registra una función; emitir llama a las funciones asociadas. En la prueba, login notifica a dos suscriptores y logout a uno.

Ejercicio 19 | Código Python

```
def crear_sistema_eventos():
    suscriptores = {}

    def suscribirse(evento, fn_callback):
        suscriptores.setdefault(evento, []).append(fn_callback)

    def emitir(evento, *args, **kwargs):
        for callback in tuple(suscriptores.get(evento, [])):
            callback(*args, **kwargs)

    def gestor(accion, evento, *args, **kwargs):
        if accion == "suscribirse":
            return suscribirse(evento, *args, **kwargs)
        if accion == "emitir":
            return emitir(evento, *args, **kwargs)
        raise ValueError("Accion desconocida: " + str(accion))

    return gestor

if __name__ == "__main__":
    bus = crear_sistema_eventos()

    bus("suscribirse", "login", lambda usuario: print(f"Bienvenido, {usuario}"))
    bus("suscribirse", "login", lambda usuario: print(f"[LOG] {usuario} inicio sesion"))
    bus("suscribirse", "logout", lambda usuario: print(f"Hasta luego, {usuario}"))

    bus("emitir", "login", "Ana")
    bus("emitir", "logout", "Ana")
```

Salida capturada al ejecutar el archivo en Python

```
Bienvenido, Ana
[LOG] Ana inicio sesion
Hasta luego, Ana
```

Los eventos sin suscriptores no producen notificaciones.

20 Consultas dinámicas sobre objetos

El primer closure conserva el campo que se consultará. Después recibe una condición y devuelve el filtro que procesa la lista. Esto permite crear consultas de productos caros y baratos usando el mismo campo precio.

Ejercicio 20 | Código Python

```
def crear_consultor(campo):
    def generar_filtro(fn_condicion_lambda):
        def filtrar(lista_objetos):
            return [obj for obj in lista_objetos if fn_condicion_lambda((obj.get(campo)
            if isinstance(obj, dict) else getattr(obj, campo, None)))]
        return filtrar
    return generar_filtro

if __name__ == "__main__":
    productos = [
        {"nombre": "Laptop", "precio": 900},
        {"nombre": "Mouse", "precio": 15},
        {"nombre": "Monitor", "precio": 250},
        {"nombre": "Teclado", "precio": 40},
    ]

    consultar_por_precio = crear_consultor("precio")
    caros = consultar_por_precio(lambda p: p > 100)
    baratos = consultar_por_precio(lambda p: p <= 40)

    print(caros(productos))
    print(baratos(productos))
```

Salida capturada al ejecutar el archivo en Python

```
[{'nombre': 'Laptop', 'precio': 900}, {'nombre': 'Monitor', 'precio': 250}]
[{'nombre': 'Mouse', 'precio': 15}, {'nombre': 'Teclado', 'precio': 40}]
```

También admite atributos de objetos. Cuando falta el campo se obtiene None y la condición debe contemplarlo.